

Why These Six Protocols

The six coordination-protocol arms are not a sample of the protocol space. Each is a response to a specific, measured failure in how the replication’s agents actually talked to each other.

1 · The evidence

We extracted every message sent in the replication’s messaging condition — **703 messages across 147 pair-runs** — coded each run’s dialogue against nine themes, and joined the dialogue to the patch-level outcome of the same run. The conflict-rate column is the share of runs exhibiting that theme which nonetheless ended in a textual merge conflict.

Theme in the replication’s messages		Prevalence	Conflict rate
T1	Coordination is file-scoped — every run names the files it will touch	100%	—
T2	Ownership is explicitly proposed (“you take X, I take Y”)	1%	—
T3	Both agents say they will append their parameter last	20%	97%
T4	Sequencing illusion (“I’ll go first”, “layer yours on top”)	24%	77%
T5	Workspace isolation discovered mid-run (“separate sandboxes”)	27%	—
T6	A clean merge is self-certified (“additive”, “no overlap”)	63%	63%
T7	A question is actually asked of the partner	9%	—
T8	Verbatim code is exchanged	3%	—
T9	Ends with a terminal “done / submitting” announcement	50%	—

Where the conflicts actually happen

The coordination that occurs is exhaustive but **file-scoped**, and that is the wrong granularity. Patch-level forensics on every conflicting run:

Patch forensics, conflicting runs	Share
Conflicting runs analysed	93
...both patches touch a shared file	100%
...the colliding file was explicitly named in the chat	100%
...collision starts at the byte-identical same line	90%
...both patches rewrite the same function signature	27%

The information required to avoid the collision was **never missing**. It was volunteered in every single run — and it was insufficient, because the collisions live **below the file**.

2 · The dialogue that proves it

A question is asked in only 9% of runs, the median run sends five messages, and the whole conversation is over within **114 seconds** — after which both agents work alone. What the pair substitutes for allocation is a shared fiction about ordering that the harness cannot honour, since each agent works in an isolated checkout. This exchange from `pallets_jinja` task 1465 is the pattern in full:

agen t1	“ack — we’re both touching <code>sync_do_groupby...</code> <i>Let’s agree on final param order/placement to avoid merge clashes.</i> ”
agen t2	“ <i>Suggest final combined order: (environment, value, attribute, default=None, separator='.', reverse=False)</i> — just append your reverse param after mine... will let you know when I’m fully done touching the file so you can layer your edit on top.”
agen t1	“my working copy doesn’t show your separator changes... we appear to be in separate sandboxes, not a shared filesystem. Since each of us submits our own <code>patch.txt</code> independently, <i>this should be fine — no actual clobbering.</i> Good luck with yours!”

Agent2 states the **correct merged signature explicitly**, and agent1 agrees to it. Neither writes it. Each emits only its own half into its own checkout, and the patches collide at `filters.py:1163`. The agreement existed as prose and never became bytes — which is why verbatim code is exchanged in only 3% of runs, while a clean merge is confidently predicted in 63%, of which 63% conflict.

The diagnosis

The protocol lets agents converge on a **description** of the merged code, while the merge is decided by the **bytes**.

The six arms therefore sweep exactly one variable: **how much of the final merged artifact the protocol obliges the pair to converge on**. Because the conflicts are measurably sub-file, this ladder makes a falsifiable prediction *in advance of the results* — arms that stop at or above file granularity should not move the merge-clean rate, and only arms that reach the construct should.

3 · The ladder, rung by rung

1. control must agree: Nothing	
Targets	— (the floor)
Rationale	Establishes the conflict rate when the channel is removed . In 63% of replication runs the pair self-certifies a clean merge (T6); if that talk were load-bearing, deleting it should hurt.
Prediction	<i>If messaging does real work, control should be clearly worse than free-text.</i>
Result	13% merge-clean, 2% both-passed.
Verdict	Free-text beats it by 8 pts, n.s. ($p = 0.105$). The talk was never load-bearing.
2. free-text must agree: Prose intent	
Targets	The replication's own condition
Rationale	Carries the replication protocol down to the screened 18-pair subset, so every other arm is measured against the same population in which T1–T9 were observed.
Prediction	<i>Reproduces the replication's themes and its floor-level merge rate.</i>
Result	21% merge-clean, 3% both-passed.
Verdict	Reference arm. Confirms the diagnosis transfers to the nano subset.
3. semi-structured must agree: Validated metadata about intent	
Targets	T1 (100%) · T2 (1%)
Rationale	The obvious first hypothesis: the coordination information is present but informal . Enforce it as typed --type / --files / --summary fields, rejected if malformed.
Prediction	<i>T1 = 100% predicts failure: agents already volunteer everything the schema asks for.</i>
Result	16% merge-clean, 3% both-passed.
Verdict	n.s. ($p = 1.00$). Rules out “they just needed to say more” — the information was never missing.

4. plan-handshake must agree: A binding partition of files	
Targets	T2 (1%) · T4 (24%, 77% conflict)
Rationale	The pair almost never allocates, and substitutes an unenforceable turn-taking story. Replace it with a mutually ACCEPTed disjoint split and a real blocking barrier (coop-awa.it) before any edit.
Prediction	<i>Fixes allocation, but the split is still file-level — so merge-clean should not move.</i>
Result	20% merge-clean, 10% both-passed.
Verdict	Merge-clean n.s. ; both-passed 2%→10% ($p = 8 \times 10^{-11}$). The barrier bought correctness, not mergeability — proving the residue is not an allocation failure.

5. designated-coder must agree: A single writer per shared file, plus a spec	
Targets	The file-granularity ceiling
Rationale	Median shared files per conflict = 1, and it is always the file both agents named — no disjoint split of it exists . So stop splitting: assign one owner, and route the other's needs as a written SPEC.
Prediction	<i>Removing the second writer should collapse the conflict rate.</i>
Result	18% merge-clean, 58% both-passed.
Verdict	Textual conflict 80%→39%. But the gain arrives as solo-rescue (43%) — one patch carrying the pair — and merge-clean stays at the floor.

6. coauthor-overlap must agree: The merged bytes themselves	
Targets	T3 (20%, 97% conflict) · T8 (3%)
Rationale	90% of conflicts start at the identical line and 27% rewrite the same def. Pairs already try to state the merged construct in prose — they just never exchange it as code. So require it verbatim, byte-for-byte, in both patches.
Prediction	<i>The only rung that reaches the construct — the only one that should work.</i>
Result	78% merge-clean, 69% both-passed.
Verdict	CMH OR 27.7, $p < 10^{-21}$. Produces 26 byte-identical merges against exactly 1 across the other five arms' 1,168 runs combined.

4 · What the ladder shows

Five of six arms fail, and they fail in the order the messaging evidence predicts. Enriching the channel (semi-structured) and planning a file-level split (plan-handshake) leave the merge-clean rate at the no-messaging floor, because neither reaches the granularity at which the patches actually collide. Designated-coder concedes the point and removes the second writer, halving textual conflict — but rescues correctness through one agent carrying the pair rather than through a clean merge. Only coauthor-overlap changes **what the merge receives** rather than how much the agents talk, and it is the only arm to separate from control on the primary endpoint after Holm correction.

Because the arms differ only in the prompt block and the message-field validation, this is a **protocol effect**, not a model or scaffold effect.

Message themes and patch forensics: `uv run python masters_thesis/protocol_analysis/replication_messages.py over logs/flash_msg_*`. Arm endpoints and inference: `analyze.py`, frozen in `data/nano_study.json`. This page: `protocol_rationale_pdf.py`.